

INTRODUCTION TO ELECTRONICS ENGINEERING CT 121

Lecture two

Overview

- Boolean Algebra
 - Boolean Arithmetic
 - Boolean Properties
 - Boolean expression
 - Circuit simplification
 - Converting truth table into Boolean expression
 - Simplification of Boolean expression and K-Mapping

Boolean Algebra

Boolean Arithmetic

- Boolean algebra, quite interestingly, is simpler than ordinary algebra.
- The differences between normal algebra and Boolean are:
 - i. In ordinary algebra, the letter symbols can take on any number of values including infinity. In Boolean algebra, they can take on either of two values, that is, 0 and 1.
 - ii. The values assigned to a variable have a numerical significance in ordinary algebra, whereas in its Boolean counterpart they have a logical significance.

- iii. While ‘.’ and ‘+’ are respectively the signs of multiplication and addition in ordinary algebra, in Boolean algebra ‘.’ means an AND operation and ‘+’ means an OR operation. For instance, $A+B$ in ordinary algebra is read as A plus B, while the same in Boolean algebra is read as A OR B. Basic logic operations such as AND, OR and NOT have already been discussed

- iv. More specifically, Boolean algebra captures the essential properties of both logic operations such as AND, OR and NOT and set operations such as intersection, union and complement. As an illustration, the logical assertion that both a statement and its negation cannot be true has a counterpart in set theory, which says that the intersection of a subset and its complement is a null (or empty) set.

Equivalent and Complement of Boolean Expressions:

- Two given Boolean expressions are said to be *equivalent* if one of them equals '1' only when the other equals '1' and also one equals '0' only when the other equals '0'.
- They are said to be the *complement* of each other if one expression equals '1' only when the other equals '0', and vice versa.

Example

- If the Boolean expression is $\bar{A}.B + A.\bar{B}$
- Then its corresponding complement is $(A + \bar{B}).(\bar{A} + B)$
- And if another Boolean expression is $(A + B).(\bar{A} + \bar{B})$
- *Then its corresponding complement is $\bar{A}.\bar{B} + A.B$*

Dual of a Boolean Expression:

- The dual of a Boolean expression is obtained by replacing all ‘.’ operations with ‘+’ operations, all ‘+’ operations with ‘.’ operations, all 0s with 1s and all 1s with 0s and leaving all literals unchanged.

Example

a. Given Boolean expression: $\bar{A}.B + A.\bar{B}$

b. Corresponding dual: $(\bar{A} + B).(A + \bar{B})$

c. Given Boolean expression: $(A + B).(\bar{A} + \bar{B})$

d. Corresponding dual: $AB + \bar{A}.\bar{B}$

Boolean Properties

- Identity elements

- $a + 0 = a$

- $a \cdot 1 = a$

- The Commutative Property:

For every a and b

- $a + b = b + a$

- $a \cdot b = b \cdot a$

- The Associative Property:

For every a , b , and c

- $a + (b + c) = (a + b) + c$

- $a \cdot (b \cdot c) = (a \cdot b) \cdot c$

- **The Distributive Property:**

$$a + (b \cdot c) = (a + b) \cdot (a + c)$$

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

- **The Existence of the Complement:**

$$a + a' = 1$$

$$a \cdot a' = 0$$

- **Absorption**

$$a + ab = a$$

$$a(a+b) = a$$

Table of Identities

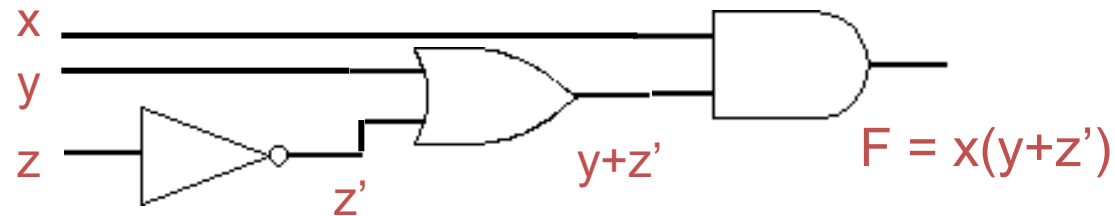
□ **TABLE 2-3**
Basic Identities of Boolean Algebra

1.	$X + 0 = X$	2.	$X \cdot 1 = X$	
3.	$X + 1 = 1$	4.	$X \cdot 0 = 0$	
5.	$X + X = X$	6.	$X \cdot X = X$	
7.	$X + \bar{X} = 1$	8.	$X \cdot \bar{X} = 0$	
9.	$\overline{\bar{X}} = X$			
10.	$X + Y = Y + X$	11.	$XY = YX$	Commutative
12.	$X + (Y + Z) = (X + Y) + Z$	13.	$X(YZ) = (XY)Z$	Associative
14.	$X(Y + Z) = XY + XZ$	15.	$X + YZ = (X + Y)(X + Z)$	Distributive
16.	$\overline{X + Y} = \bar{X} \cdot \bar{Y}$	17.	$\overline{X \cdot Y} = \bar{X} + \bar{Y}$	DeMorgan's

Boolean Functions

- Boolean algebra deals with binary variables and logic operations.
- Function results in binary 0 or 1

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

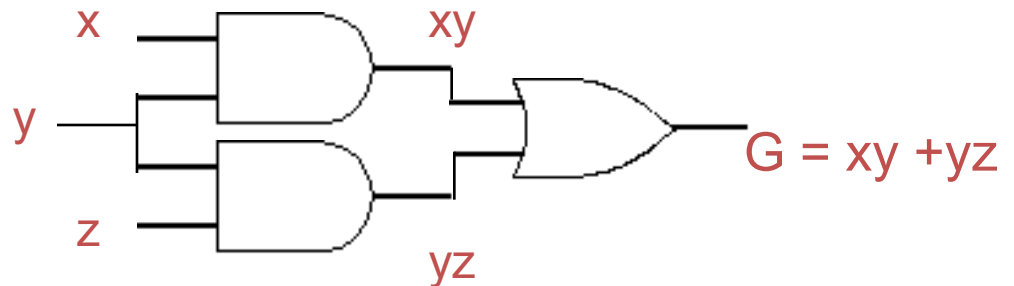


$$F = x(y+z')$$

Boolean Functions

- Boolean algebra deals with binary variables and logic operations.
- Function results in binary 0 or 1

x	y	z	xy	yz	G
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	1	1
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	1	0	1
1	1	1	1	1	1



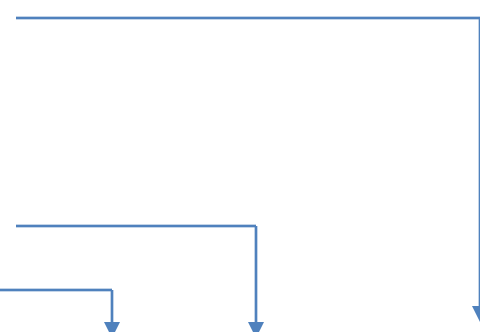
We will learn how to transition between equation, circuits (symbols), and truth table.

Truth Table to Expression

- Converting a truth table to an expression
 - Each row with output of **1** becomes a **product term**
 - **Sum product** terms together.

x	y	z	G
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

*Any Boolean Expression can be represented in **sum of products** form!*

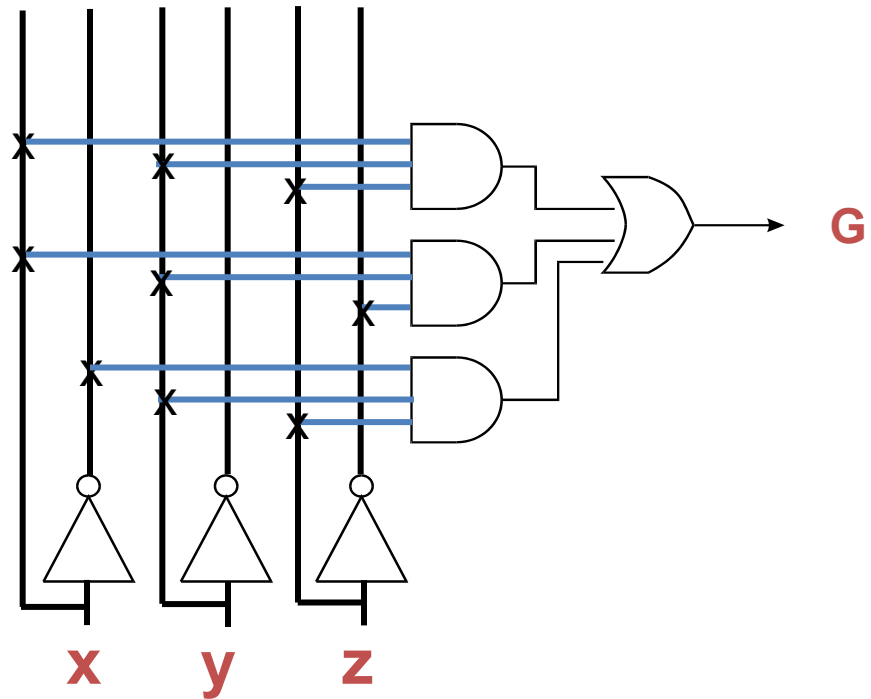

$$xyz + xyz' + x'yz$$

Equivalent Representations of Circuits

- All three formats are equivalent
- Number of 1's in truth table output column equals AND terms for Sum-of-Products (SOP)

x	y	z	G
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$G = xyz + xyz' + x'yz$$



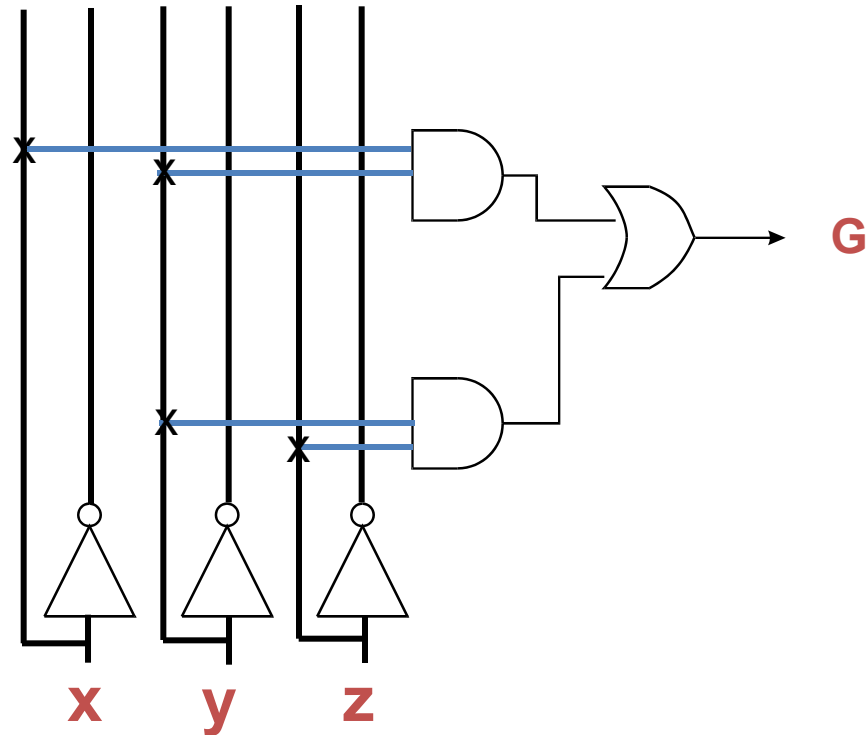
Reducing Boolean Expressions

- Is this the smallest possible implementation of this expression? **No!** $G = xyz + xyz' + x'yz$
- Use Boolean Algebra rules to reduce complexity while preserving functionality.
- Step 1: Use Theorem 1 ($a + a = a$)
 - So $xyz + xyz' + x'yz = xyz + xyz + xyz' + x'yz$
- Step 2: Use distributive rule $a(b + c) = ab + ac$
 - So $xyz + xyz + xyz' + x'yz = xy(z + z') + yz(x + x')$
- Step 3: Use Postulate 3 ($a + a' = 1$)
 - So $xy(z + z') + yz(x + x') = xy.1 + yz.1$
- Step 4: Use Postulate 2 ($a . 1 = a$)
 - So $xy.1 + yz.1 = xy + yz = xyz + xyz' + x'yz$

Reduced Hardware Implementation

- Reduced equation requires less hardware!
- Same function implemented!

x	y	z	G
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



$$G = xyz + xyz' + x'yz = xy + yz$$

Canonical Forms (Minterms and Maxterms)

- Each variable in a Boolean expression is a **literal**
- Boolean variables can appear in normal (**x**) or complement form (**x'**)
- Each AND combination of terms is a **minterm**
- Each OR combination of terms is a **maxterm**

For example:

Minterms

x	y	z	Minterm	
0	0	0	$x'y'z'$	m_0
0	0	1	$x'y'z$	m_1
...				
1	0	0	$xy'z'$	m_4
...				
1	1	1	xyz	m_7

For example:

Maxterms

x	y	z	Maxterm	
0	0	0	$x+y+z$	M_0
0	0	1	$x+y+z'$	M_1
...				
1	0	0	$x'+y+z$	M_4
...				
1	1	1	$x'+y'+z'$	M_7

Representing Functions with Minterms

- Minterm number same as row position in truth table (starting from top from 0)
- Shorthand way to represent functions

x	y	z	G
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$G = xyz + xyz' + x'yz$$



$$G = m_7 + m_6 + m_3 = \Sigma(3, 6, 7)$$

Complementing Functions

- Minterm number same as row position in truth table (starting from top from 0)
- Shorthand way to represent functions

x	y	z	G	G'
0	0	0	0	1
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	1
1	1	0	1	0
1	1	1	1	0

$$G = xyz + xyz' + x'yz$$

$$G' = (xyz + xyz' + x'yz)' =$$

Can we find a simpler representation?

Complementing Functions

- Step 1: assign temporary names

- $b + c = z$

- $(a + z)' = G'$

$$G = a + b + c$$

$$G' = (a + b + c)'$$

- Step 2: Use DeMorgans' Law

- $(a + z)' = a' \cdot z'$

- Step 3: Resubstitute $(b+c)$ for z

- $a' \cdot z' = a' \cdot (b + c)'$

- Step 4: Use DeMorgans' Law

- $a' \cdot (b + c)' = a' \cdot (b' \cdot c')$

$$G = a + b + c$$

$$G' = a' \cdot b' \cdot c' = a'b'c'$$

- Step 5: Associative rule

- $a' \cdot (b' \cdot c') = a' \cdot b' \cdot c'$

Complementation Example

- Find complement of $F = x'z + yz$
 - $F' = (x'z + yz)'$
- DeMorgan's
 - $F' = (x'z)' (yz)'$
- DeMorgan's
 - $F' = (x'' + z')(y' + z')$
- Reduction = eliminate double negation on x
 - $F' = (x + z')(y' + z')$



This format is called product of sums

Conversion Between Canonical Forms

- Easy to convert between minterm and maxterm representations
- For maxterm representation, select rows with 0's

x	y	z	G
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$G = xyz + xyz' + x'yz$$



$$G = m_7 + m_6 + m_3 = \Sigma(3, 6, 7)$$



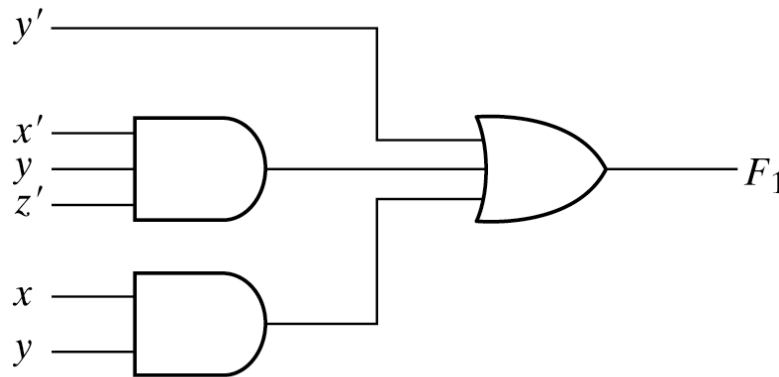
$$G = M_0 M_1 M_2 M_4 M_5 = \Pi(0, 1, 2, 4, 5)$$



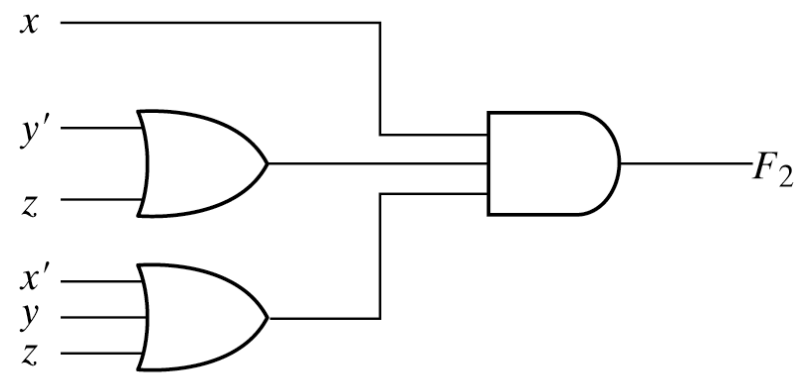
$$G = (x+y+z)(x+y+z')(x+y'+z)(x'+y+z)(x'+y+z')$$

Representation of Circuits

- All logic expressions can be represented in 2-level format
- Circuits can be reduced to minimal 2-level representation
- Sum of products representation most common



(a) Sum of Products



(b) Product of Sums

Fig. 2-3 Two-level implementation

Karnaugh maps

- Alternate way of representing Boolean function
 - All rows of truth table represented with a square
 - Each square represents a minterm
- Easy to convert between truth table, K-map, and SOP
 - Unoptimized form: number of 1's in K-map equals number of minterms (products) in SOP
 - Optimized form: reduced number of minterms

Check
example of
function
with the
given truth
table

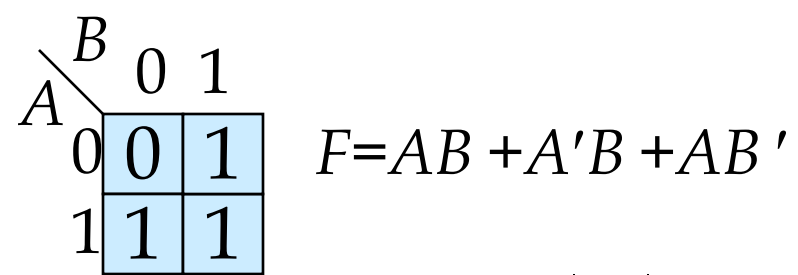
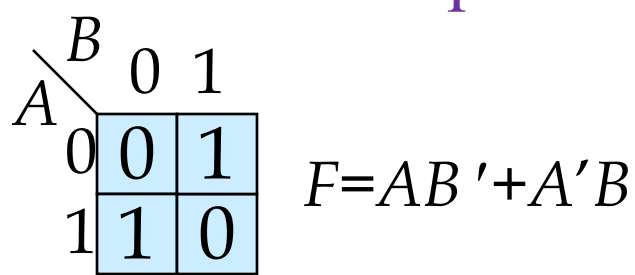
x	y	F
0	0	1
0	1	1
1	0	0
1	1	0

$$F = \Sigma(m_0, m_1) = x'y + x'y'$$

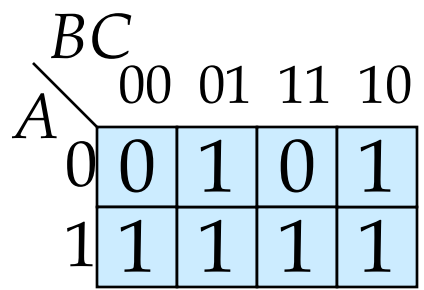
		y	
		0	1
x	0	$x'y'$	$x'y$
	1	xy'	xy

		y	
		0	1
x	0	1	1
	1	0	0

- A Karnaugh map is a graphical tool for assisting in the general simplification procedure.
- Two variable maps.



◦ **Three variable maps.**



A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$$F = AB'C' + AB'C + ABC + ABC' + A'B'C + A'BC'$$

Rules for K-Maps

- We can reduce functions by circling 1's in the K-map
- Each circle represents minterm reduction
- Following circling, we can deduce minimized and-or form.

Rules to consider

- Every cell containing a 1 must be included at least once.
- The largest possible “power of 2 rectangle” must be enclosed.
- The 1's must be enclosed in the smallest possible number of rectangles.

Karnaugh maps

- Numbering scheme based on Gray-code
 - e.g., 00, 01, 11, 10
 - Only a single bit changes in code for adjacent map cells
 - This is necessary to observe the variable transitions

		A	
		1	1
C		1	1
	B		

$$G(A,B,C) = A$$

	AB	A			
		00	01	11	10
C	0				
	1				
		B			

	A			
	1	0	0	1
C	0	0	1	1
	B			

$$F(A,B,C) = \sum m(0,4,5,7) = AC + B'C'$$

More Karnaugh Map Examples

- Examples

		a	
		0	1
b	0	0	1
	1	0	1

$f = a$

		a	
		0	1
b	0	1	1
	1	0	0

$g = b'$

		ab			
		00	01	11	10
c	0	0	0	1	0
	1	0	1	1	1

$\text{cout} = ab + bc + ac$

		ab			
		00	01	11	10
c	0	0	0	1	1
	1	0	0	1	1

$f = a$

1. Circle the largest groups possible.
2. Group dimensions must be a power of 2.
3. Remember what circling means!

Summary

- Karnaugh map allows us to represent functions with new notation
- Representation allows for logic reduction.
 - Implement same function with less logic
- Each square represents one minterm
- Each circle leads to one product term
- Not all functions can be reduced
- Each circle represents an application of:
 - Distributive rule -- $x(y + z) = xy + xz$
 - Complement rule -- $x + x' = 1$

Karnaugh Maps and Don't Cares

- Karnaugh maps with four inputs
 - Same basic rules as three input K-maps
- Understanding prime implicants
 - Related to minterms
- Covering all implicants
- Using **Don't Cares** to simplify functions
 - Don't care outputs are undefined
- Summarizing Karnaugh maps

Karnaugh Maps for Four Input Functions

- Represent functions of 4 inputs with 16 minterms
- Use same rules developed for 3-input functions
- Note bracketed sections shown in example.

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

(a)

		yz		y		
		0 0	0 1	1 1	1 0	
w	x	0 0	$w'x'y'z'$	$w'x'y'z$	$w'x'yz$	$w'x'yz'$
	0 1	$w'xy'z'$	$w'xy'z$	$w'xyz$	$w'xyz'$	} x
	1 1	$wxy'z'$	$wxy'z$	$wxyz$	$wxyz'$	
	1 0	$wx'y'z'$	$wx'y'z$	$wx'yz$	$wx'yz'$	
		z				

(b)

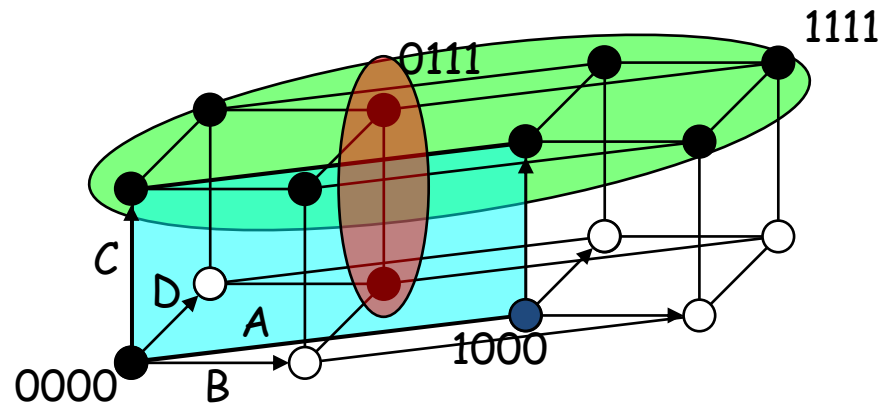
Fig. 3-8 Four-variable Map

Karnaugh map: 4-variable example

- $F(A,B,C,D) = \sum m(0,2,3,5,6,7,8,10,11,14,15)$

$$F = C + A'BD + B'D'$$

				A	
C	1	0	0	1	D
	0	1	0	0	
	1	1	1	1	
	1	1	1	1	
				B	



Solution set can be considered as a coordinate System!

Design examples

				A
	0	0	0	0
	1	0	0	0
	1	1	0	1
C	1	1	0	0
				B

K-map for LT

				A
	1	0	0	0
	0	1	0	0
	0	0	1	0
C	0	0	0	1
				B

K-map for EQ

				A
	0	1	1	1
	0	0	1	1
	0	0	0	0
C	0	0	1	0
				B

K-map for GT

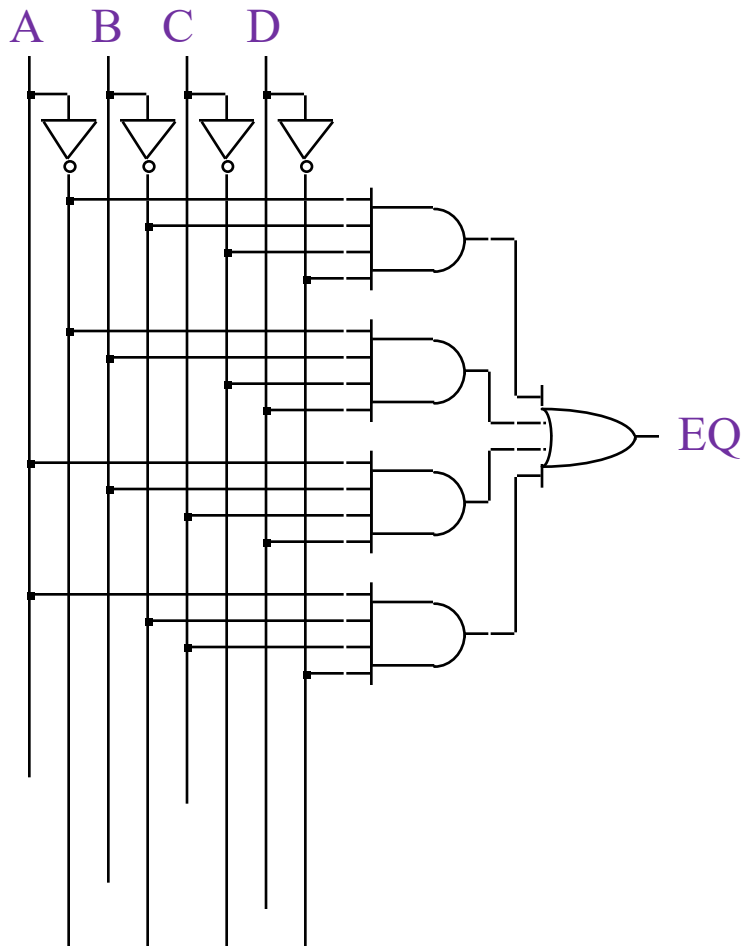
$$LT = A'B'D + A'C + B'CD$$

$$EQ = A'B'C'D' + A'BC'D + ABCD + AB'CD'$$

$$GT = BC'D' + AC' + ABD'$$

Can you draw the truth table for these examples?

Physical Implementation



- Step 1: Truth table
- Step 2: K-map
- Step 3: Minimized sum-of-products
- Step 4: Physical implementation with gates

				A
C	1	0	0	0
	0	1	0	0
	0	0	1	0
	0	0	0	1
				B

K-map for EQ

Karnaugh Maps

- Four variable maps.

		<i>CD</i>			
		00	01	11	10
<i>AB</i>	00	0	0	0	1
	01	1	1	0	1
	11	1	1	1	1
	10	1	0	1	1

$$F = A'BC' + A'CD' + ABC$$

$$+ AB'C'D' + ABC' + AB'C$$

$$F = BC' + CD' + AC + AD'$$

- Need to make sure all 1's are covered
- Try to minimize total product terms.
- Design could be implemented using NANDs and NORs

Karnaugh maps: Don't cares

- In some cases, outputs are undefined
- We “don't care” if the logic produces a 0 or a 1
- This knowledge can be used to simplify functions.

		A			
		00	01	11	10
C	00	0	0	X	0
	01	1	1	X	1
	11	1	1	0	0
	10	0	X	0	0

- Treat X's like either 1's or 0's
- Very useful
- OK to leave some X's uncovered

Karnaugh maps: Don't cares

- $f(A,B,C,D) = \square\square m(1,3,5,7,9) + d(6,12,13)$
– without don't cares

- $f =$
 $A'D + C'D$

		<u>A</u>			
		AB			
		00	01	11	10
CD	00	0	0	X	0
	01	1	1	X	1
	11	1	1	0	0
	10	0	X	0	0
		<u>B</u>			
		D			
		C			

A	B	C	D	f
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	X
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	X
1	1	0	1	X
1	1	1	0	0
1	1	1	1	0

Don't Care Conditions

- In some situations, we don't care about the value of a function for certain combinations of the variables.
 - these combinations may be impossible in certain contexts
 - or the value of the function may not matter in when the combinations occur
- In such situations we say the function is *incompletely specified* and there are multiple (completely specified) logic functions that can be used in the design.
 - so we can select a function that gives the simplest circuit
- When constructing the terms in the simplification procedure, we can choose to either cover or not cover the don't care conditions.

Map Simplification with Don't Cares

$AB \backslash CD$		CD			
		00	01	11	10
00	00	0	1	0	0
01	01	x	x	x	1
11	11	1	1	1	x
10	10	x	0	1	1

$$F = A'C'D + B + AC$$

◦ **Alternative covering.**

$AB \backslash CD$		CD			
		00	01	11	10
00	00	0	1	0	0
01	01	x	x	x	1
11	11	1	1	1	x
10	10	x	0	1	1

$$F = A'B'C'D + ABC' + BC + AC$$

Karnaugh maps: don't cares (cont'd)

- $f(A,B,C,D) = \sum m(1,3,5,7,9) + d(6,12,13)$
 - $f = A'D + B'C'D$ without don't cares
 - $f = A'D + C'D$ with don't cares

				A	
		0	0	X	0
		1	1	X	1
		1	1	0	0
		0	X	0	0
		B			
C					
		D			

by using don't care as a "1"
a 2-cube can be formed
rather than a 1-cube to cover
this node

don't cares can be treated as
1s or 0s
depending on which is more
advantageous